KONSEP INTI

Satu proses, beberapa alur kerja

Materi kelas dengan contoh baru. Ikuti proses yang mengeksekusi setiap baris, lalu cocokkan dengan bukti praktik.

fork()

Membentuk child baru. Parent dan child melanjutkan setelah pemanggilan dengan nilai kembali berbeda. Variabel biasa berada pada ruang memori masing-masing. [1]

execlp()

Mengganti program pada proses pemanggil. PID tetap; keberhasilan tidak kembali ke kode lama. Nama tanpa slash dicari melalui PATH. Kegagalan mengembalikan -1. [2]

wait(NULL)

Parent mengambil satu child yang berakhir. Jika perlu, parent menunggu. Nilai kembali sukses adalah PID child; -1 berarti gagal. NULL tidak menyimpan status akhir. [3]

Mengenali cabang setelah fork()

nilai > 0

Parent menerima PID child.

nilai == 0

Child menerima penanda nol.
Penanda ini bukan PID asli child.

nilai == -1

Gagal membentuk child.
Pisahkan cabang kegagalan.

Tiga pertanyaan saat membaca kode

Proses mana yang mencapai baris ini? Apakah proses itu sudah berhenti atau berganti program? Apakah ada wait yang membatasi kapan parent dapat melanjutkan?

Ruang lingkup C: printf(), fork(), execlp(), wait(), dan return. Tidak ada tambahan API C pada contoh. [1]-[4] adalah manual resmi Linux yang tercantum pada halaman 8.

Dijalankan pada Linux melalui PTY

Kompilasi dan keluaran di bawah berasal dari eksekusi aktual, bukan prediksi yang ditulis menjadi hasil.

Lingkungan Linux + terminal PTY

LINUX / PTY | TRANSKRIP EKSEKUSI AKTUAL

```
$ uname -srm
Linux 6.6.87.2-microsoft-standard-WSL2 x86_64
$ cat /etc/alpine-release
3.23.6
$ gcc --version | head -1
gcc (Alpine 15.2.0) 15.2.0
$ test -t 1 && echo stdout=terminal-PTY
stdout=terminal-PTY
$ command -v printf
printf
$ /usr/bin/printf "%s\n" executable-tersedia
executable-tersedia
```

00_lingkungan.pty.log | 13 Oktober 2026

Screenshot halaman browser lokal yang menampilkan transkrip PTY asli. Program C menulis ke terminal Linux; sesudahnya, teks hasil ditampilkan untuk ditangkap sebagai gambar.

Lingkungan

Host Windows; kernel Linux WSL2 pada distribusi docker-desktop. Compiler dan libc Alpine berada dalam chroot terisolasi di drive D. Docker daemon tidak diperlukan.

Pemeriksaan

stdout=terminal-PTY membuktikan keluaran program terhubung ke terminal virtual. Semua contoh memakai newline pada printf. Ini menghindari perbedaan dasar akibat penangkapan langsung melalui pipe. [4]

Menjalankan ulang

Di terminal Linux, buat folder bin. Kompilasi setiap source dengan GCC dan flag pada screenshot, lalu jalankan binarinya langsung. Program 03 memerlukan executable printf pada PATH.

Jejak yang disimpan

Source C, binary Linux, log mentah PTY, teks keluaran, screenshot, serta metadata waktu dan hash tersimpan bersama modul. PID dapat berubah setiap kali dijalankan.

Waktu eksekusi: 2026-10-13 10:17:38 (Asia/Bangkok)

DEMONSTRASI 01

Loket label: tugas berbeda, alur berbeda

Child menyelesaikan label lalu return. Parent menunggu sebelum menulis laporan penutup.

Source: src/01_loket_label.c

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    printf("Loket label dibuka.\n");
    pid_t tugas = fork();

    if (tugas < 0) {
        printf("Petugas tambahan tidak terbentuk.\n");
        return 1;
    }
    if (tugas == 0) {
        printf("Petugas: label paket K-31 siap.\n");
        return 0;
    }

    printf("Pengawas: PID petugas = %ld\n", (long)tugas);
    if (wait(NULL) < 0) {
        printf("Pengawas: penantian gagal.\n");
        return 2;
    }
    printf("Pengawas: petugas telah berakhir.\n");
    return 0;
}
```

Cabang tugas < 0 disediakan, tetapi kegagalan fork tidak dipaksakan dalam eksperimen ini.

Loket label

LINUX / PTY | TRANSKRIP EKSEKUSI AKTUAL

```
$ gcc -std=c11 -Wall -Wextra -Werror -pedantic \
  src/01_loket_label.c -o bin/01_loket_label
Kompilasi: OK (tanpa warning)
$ ./bin/01_loket_label
Loket label dibuka.
Pengawas: PID petugas = 28
Petugas: label paket K-31 siap.
Pengawas: petugas telah berakhir.
Status keluar program: 0
```

01_loket_label.pty.log | 13 Oktober 2026

Screenshot hasil run pertama; nomor PID adalah nilai yang benar-benar teramati.

Cara menelusuri

Pesan pembukaan terjadi sebelum child terbentuk. Child masuk ke cabang tugas == 0 dan berhenti pada return; laporan penutup hanya dicapai parent.

Hubungan urutan

Petugas telah berakhir sebelum parent melewati wait yang berhasil. Pesan PID parent dan pesan tugas child dapat saling mendahului; screenshot hanya menunjukkan satu jadwal.

DEMONSTRASI 02

Dua petugas dibuat oleh satu pengawas

Hanya parent mencapai fork kedua. Dua wait mengambil dua child, satu per pemanggilan yang berhasil.

Source: src/02_dua_petugas.c

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    pid_t petugas = fork();
    if (petugas < 0) {
        printf("Pembentukan petugas label gagal.\n");
        return 1;
    }
    if (petugas == 0) {
        printf("Petugas label: paket K-31 siap.\n");
        return 0;
    }
    printf("Pengawas: PID label = %ld\n", (long)petugas);

    petugas = fork();
    if (petugas < 0) {
        printf("Pembentukan petugas arsip gagal.\n");
        if (wait(NULL) < 0) return 2;
        return 1;
    }
    if (petugas == 0) {
        printf("Petugas arsip: dokumen R-18 siap.\n");
        return 0;
    }
    printf("Pengawas: PID arsip = %ld\n", (long)petugas);

    for (int urutan = 1; urutan <= 2; urutan++) {
        pid_t selesai = wait(NULL);
        if (selesai < 0) {
            printf("Pengawas: penantian gagal.\n");
            return 2;
        }
        printf("Diambil ke-%d: PID %ld\n", urutan, (long)selesai);
    }
    printf("Pengawas: kedua petugas telah berakhir.\n");
    return 0;
}
```

Dua petugas

LINUX / PTY | TRANSKRIP EKSEKUSI AKTUAL

```
$ gcc -std=c11 -Wall -Wextra -Werror -pedantic \
  src/02_dua_petugas.c -o bin/02_dua_petugas
Kompilasi: OK (tanpa warning)
$ ./bin/02_dua_petugas
Pengawas: PID label = 40
Petugas label: paket K-31 siap.
Pengawas: PID arsip = 41
Diambil ke-1: PID 40
Petugas arsip: dokumen R-18 siap.
Diambil ke-2: PID 41
Pengawas: kedua petugas telah berakhir.
Status keluar program: 0
```

02_dua_petugas.pty.log | 13 Oktober 2026

Cocokkan PID label/arsip dengan PID yang dikembalikan oleh setiap wait.

Pohon proses

Parent membuat dua child. Setiap child return sebelum mencapai pekerjaan parent berikutnya. Saat keduanya berhasil terbentuk, ada tiga proses yang terlibat.

Yang dapat berubah

wait(NULL) dapat mengambil salah satu child yang sudah berakhir. Urutan PID yang diambil tidak wajib sama dengan urutan pembentukan. Laporan akhir menunggu kedua wait berhasil.

DEMONSTRASI 03

Berganti program tanpa membuat child

Contoh ini langsung memanggil execlp. Tidak ada fork pada source ini.

Source: src/03_ganti_program.c

```c
#include <stdio.h>
#include <unistd.h>

int main(void) {
    printf("Penulis laporan: menyerahkan kendali.\n");

    execlp("printf", "printf", "Kode pekerjaan: %s\n",
           "L-42", (char *)NULL);

    printf("Penulis laporan: pergantian gagal.\n");
    return 2;
}
```

Satu proses, program berubah [2]

Sebelum

Proses menjalankan penulis laporan.

↓

Sesudah

Proses yang sama menjalankan executable printf.

PID tetap merupakan sifat execlp yang berhasil menurut manual. Screenshot keluaran ini tidak mengukur PID secara langsung.

Pergantian program berhasil

LINUX / PTY | TRANSKRIP EKSEKUSI AKTUAL

```
$ gcc -std=c11 -Wall -Wextra -Werror -pedantic \
  src/03_ganti_program.c -o bin/03_ganti_program
Kompilasi: OK (tanpa warning)
$ ./bin/03_ganti_program
Penulis laporan: menyerahkan kendali.
Kode pekerjaan: L-42
Status keluar program: 0
```

03_ganti_program.pty.log | 13 Oktober 2026

Keluaran Kode pekerjaan berasal dari executable printf pengganti, bukan printf C setelah execlp.

Membaca argumen

Nama printf pertama adalah executable yang dicari pada PATH. Nama berikutnya menjadi argv[0]. Format dan kode pekerjaan adalah argumen program tersebut; (char *)NULL menutup daftar.

Setelah keberhasilan

Baris pergantian gagal tidak tercetak. Program lama sudah diganti; kode itu tidak dilanjutkan ketika executable pengganti selesai. Status 0 di screenshot adalah status program pengganti.

DEMONSTRASI 04

Pergantian gagal: kode lama berlanjut

Jalur lokal diperiksa tidak tersedia sebelum program dijalankan. Percobaan ini sengaja menghasilkan status keluar 2.

Source: src/04_gagal_ganti.c

```c
#include <stdio.h>
#include <unistd.h>

int main(void) {
    printf("Penulis laporan: mencoba alat lokal.\n");

    execlp("./alat_laporan_tidak_tersedia",
           "alat_laporan_tidak_tersedia", (char *)NULL);

    printf("Penulis laporan: kembali karena exec gagal.\n");
    return 2;
}
```

Pergantian program gagal

LINUX / PTY | TRANSKRIP EKSEKUSI AKTUAL

```
$ gcc -std=c11 -Wall -Wextra -Werror -pedantic \
  src/04_gagal_ganti.c -o bin/04_gagal_ganti
Kompilasi: OK (tanpa warning)
$ test ! -e ./alat_laporan_tidak_tersedia
Pemeriksaan: alat lokal memang tidak tersedia
$ ./bin/04_gagal_ganti
Penulis laporan: mencoba alat lokal.
Penulis laporan: kembali karena exec gagal.
Status keluar program: 2
```

04_gagal_ganti.pty.log | 13 Oktober 2026

Log juga menyimpan pemeriksaan bahwa executable lokal memang tidak ada.

Pisahkan tiga hal

Pemanggilan fungsi

execlp gagal kembali ke kode lama dengan -1.

Pesan keluaran

Pesan kembali dicetak oleh program lama.

Status program

return 2 diterima shell sebagai status keluar 2.

Alur yang terjadi

execlp tidak dapat menjalankan berkas itu dan kembali dengan -1. printf sesudahnya lalu dijalankan, kemudian return 2 mengakhiri program lama.

Makna status 2

Angka 2 dipilih oleh source ini untuk menandai jalur gagal. Angka tersebut bukan nilai kembali execlp; nilai kembali fungsi pada kegagalan adalah -1. [2]

Keberhasilan wait(NULL) pada contoh dengan child hanya memastikan child telah berakhir; status tugasnya tidak disimpan. [3]

PENGAMATAN ULANG

Ulangi hasil, periksa batas urutan

Demonstrasi 01 dan 02 masing-masing dijalankan lima kali lagi pada PTY yang sama.

PID yang teramati pada demonstrasi 02

Run	PID label	PID arsip	wait ke-1	wait ke-2
1	79	80	79	80
2	82	83	82	83
3	85	86	85	86
4	88	89	88	89
5	91	92	91	92

Hasil verifikasi

Empat source lolos kompilasi dengan -Wall -Wextra -Werror -pedantic. Demo 01-03 berakhir dengan status 0; demo 04 berakhir dengan status 2 yang memang dirancang.

Sepuluh run tambahan

Semua run tambahan selesai dengan status 0. Demo 01 mempertahankan batas child sebelum penutup. Demo 02 mengambil tepat dua PID child sebelum penutup.

Batas bukti

Data ini memeriksa contoh dan lingkungan yang dipakai. Lima pengulangan tidak membuktikan semua jadwal yang mungkin; jaminan urutan tetap ditentukan oleh alur kode.

Kenapa memakai terminal? [4]

stdout yang diarahkan ke file atau pipe dapat memiliki penyanggaan berbeda. Teks yang belum dikeluarkan sebelum fork dapat ikut tersalin; isi buffer lama sebelum execlp berhasil dapat hilang. Newline dan PTY menjaga fokus percobaan pada alur proses.

Logs 05_pengulangan.pty.log mempertahankan byte keluaran asli. Versi .txt hanya menormalkan akhir baris untuk dibaca. Pemeriksaan tidak memaksa kegagalan fork atau menambah fungsi C.

DISKUSI KELAS DAN SUMBER

Tandai alur, amati, lalu jelaskan

Pertanyaan berikut memakai demonstrasi baru dalam modul dan tidak disertai kunci jawaban ujian.

D1 Pada eksperimen petugas label, buat gambar alur dan tandai titik masuk cabang serta titik return. Bagaimana penandaan itu membantu menjelaskan bagian mana yang masih dapat dijalankan oleh setiap proses?

D2 Untuk eksperimen dua petugas, rancang tabel pengamatan yang mencatat PID dari setiap fork pada parent dan PID dari setiap wait. Data apa yang perlu dibandingkan ketika eksperimen diulang?

D3 Pada eksperimen pergantian program langsung, tentukan bukti dari kode yang akan kamu gunakan untuk menjelaskan hubungan antara program semula dan program pengganti. Bedakan bukti itu dari hal yang hanya terlihat pada tampilan terminal.

D4 Jika jalur executable lokal sengaja menunjuk ke berkas yang tidak tersedia, susun langkah pemeriksaan untuk menentukan apakah pesan terminal berasal dari program pengganti atau dari jalur penanganan kegagalan di program semula.

D5 Dua rekaman eksperimen petugas menunjukkan urutan pesan yang berbeda. Bagaimana kamu menentukan hubungan urutan yang diwajibkan oleh kode, dan pengamatan tambahan apa yang berguna sebelum membuat kesimpulan?

Pola praktik

Baca source dan gambar alurnya sebelum menjalankan program. Simpan prediksi sendiri, bandingkan dengan keluaran, lalu jelaskan batas yang dijamin kode.

Pegangan saat berdiskusi

- Child melanjutkan setelah fork, dengan penanda kembali nol.
- return dari main mengakhiri proses yang menjalankannya.
- Satu wait yang berhasil mengambil satu child.
- execlp yang berhasil mengganti program dan tidak kembali.
- wait(NULL) tidak menyimpan status akhir tugas child.

Sumber primer Linux

[1] fork(2) - Linux man-pages

[2] exec(3) - Linux man-pages

[3] wait(2) - Linux man-pages

[4] stdio(3) - Linux man-pages

Klik nama manual untuk membuka sumber. Diperiksa pada 13 Oktober 2026. Materi dibuat untuk pengajaran konsep dengan contoh yang berbeda.

REPOSITORI PRAKTIK

Kode dan bahan praktik di GitHub

Ambil source, lihat screenshot, dan buka modul terbaru melalui repositori OSExercise.

SeedFlora / OSExercise

https://github.com/SeedFlora/OSExercise

Ambil dan jalankan di terminal Linux

```
$ git clone https://github.com/SeedFlora/OSExercise.git
$ cd OSExercise
$ mkdir -p bin
$ gcc -std=c11 -Wall -Wextra -Werror -pedantic \
  src/01_loket_label.c -o bin/01_loket_label
$ ./bin/01_loket_label
```

Perintah di atas adalah panduan reproduksi. Lanjutkan dengan source 02, 03, dan 04. Gunakan terminal langsung agar pengamatan alur tidak tercampur perubahan buffering. Program 04 sengaja menghasilkan status 2.

Bahan dalam repositori

src/: empat program C; screenshots/: lima tangkapan transkrip; logs/: bukti keluaran dan verifikasi; output/pdf/: modul ini; tools/: sumber penyusunan materi.

Pindai untuk membuka GitHub

Buka PDF terbaru di GitHub

Klik banner repositori atau tombol PDF untuk membuka bahan daring.